Báo cáo kết quả huấn luyện & Thử nghiệm

Đề tài: Phát hiện trùng lặp mã nguồn C/C++ và Python

Ngày 16 tháng 10 năm 2026

1 Kết quả tuần này

Hoàn thành train 5 phương pháp trên POJ-104: CodeBERT (`codebert-base`, 3 epoch), GraphCodeBERT (`graphcodebert-base`, 3 epoch), ASTNN (7 epoch), CPG-GNN (15 epoch), Hybrid (warm-start từ GraphCodeBERT và CPG-GNN, fine-tune 4 epoch). Ngưỡng chọn trên validation, giữ nguyên khi test và khi demo. Minh hoạ ở mục 6 chuyển sang bộ test tự tạo `data/custom_suite` (4 bài riêng biệt, mỗi bài có biến thể đổi vòng lặp/IO/đổi tên + non-clone khó), thay cho 2 cặp POJ-104 cũ.

Ngày 12-10 chạy thêm M1-512 và M2-512 (warm-start từ checkpoint 256, `max_length` 512, effective batch 32 giữ nguyên, 2 epoch do hết budget 9h): test F1 0,941 và 0,955, đều vượt bản 256 (0,910 và 0,907). Chi tiết ở mục 2b và 3b. Ngày 15-10 chạy thêm M5-512 (Hybrid trên nền encoder 512: text M2-512 giữ nguyên graph M4, 2 epoch do hết budget, best epoch 0): test F1 0,952, đứng thứ hai toàn đoàn sau M2-512.

2 Kết quả huấn luyện

Val F1 theo epoch (ngưỡng chọn trên validation).

Epoch	M3	M4	M1-256	M2-256	M1-512	M2-512	M5-256	M5-512
E0	0,789	0,782	0,919	**0,915**	**0,921**	0,932	**0,914**	**0,915**
E1	0,814	0,773	**0,919**	0,914	0,918	**0,933**	0,893	0,900
E2	0,841	0,806	0,912	0,911	—	—	0,859	—
E3	**0,853**	0,794	—	—	—	—	0,879	—
E4	0,804	0,833	—	—	—	—	—	—
E5	0,823	0,816	—	—	—	—	—	—
E6	0,841	0,804	—	—	—	—	—	—
E7	—	0,817	—	—	—	—	—	—
E8	—	0,816	—	—	—	—	—	—
E9	—	0,836	—	—	—	—	—	—
E10	—	0,827	—	—	—	—	—	—
E11	—	0,830	—	—	—	—	—	—
E12	—	0,837	—	—	—	—	—	—
E13	—	0,838	—	—	—	—	—	—
E14	—	**0,842**	—	—	—	—	—	—

M1 = CodeBERT, M2 = GraphCodeBERT, M3 = ASTNN, M4 = CPG-GNN, M5 = Hybrid. Cột M1/M2-512 warm-start từ checkpoint 256 (12-10, T4, hết budget 9h nên dừng ở 2 epoch); cột M5-512 là Hybrid trên nền encoder 512 (15-10, hết budget nên dừng ở 2 epoch, best epoch 0). Nguồn: `training_history.csv` trong `results/exp00{2,3,6}_*_512/`.

3 Kết quả test

PP	Prec.	Rec.	F1	AUC	Ngưỡng
M3	0,760	0,894	0,821	0,889	0,557
M4	0,723	0,887	0,797	0,882	0,470
M1-256	0,931	0,890	0,910	0,973	0,590
M2-256	0,883	0,933	0,907	0,971	0,430
M1-512	0,947	0,935	**0,941**	**0,987**	0,210
M2-512	0,951	0,959	**0,955**	**0,991**	0,660
M5-256	0,905	0,922	**0,913**	0,970	0,370
M5-512	0,961	0,942	**0,952**	0,988	0,010

3.1 So sánh 256 với 512 token

M1: F1 0,910 → **0,941** (+0,031); FP 363 → 288, FN 606 → 357; val F1 0,920 → 0,924 (ngưỡng 0,21). M2: F1 0,907 → **0,955** (+0,048); FP 682 → 269, FN 371 → 228; val F1 0,916 → 0,934 (ngưỡng 0,66). M5: F1 0,913 → **0,952** (+0,039); val F1 0,915 → 0,938 (15-10, best epoch 0 rồi overfit như bản 256). Lưu ý: ngưỡng M5-512 rất mỏng (0,01, điểm phân cực về 0/1) nên khi dùng demo cần theo dõi verdict, không chỉ nhìn điểm số. Tỉ lệ code bị cắt giảm từ 78,6% (train) / 73,5% (val) xuống còn 25,5% / 22,3%, đúng vùng lỗi FN tập trung ở code dài nên recall tăng mạnh nhất (M1 0,890 → 0,935; M2 0,933 → 0,959). Cả hai bản 512 warm-start từ checkpoint 256, chỉ chạy 2 epoch (hết budget 9h trên T4: M1 best epoch 0 với val F1 0,921; M2 best epoch 1 với val F1 0,933). Nguồn số liệu: `results/exp002_codebert_poj104_512/metrics.json` và `results/exp003_graphcodebert_poj104_512/metrics.json`.

6 Minh hoạ

Cách đọc hình:

- Hai phía cùng màu là hai đoạn chính mô hình đó cho là khớp nhau.
- Mỗi mô hình một bảng màu riêng vì mỗi họ nhìn code bằng tín hiệu riêng.
- CodeBERT, GraphCodeBERT, Hybrid bắt cặp từng từ trên vector encoder.
- ASTNN bắt cặp câu lệnh trên vector RvNN, bỏ span bao hơn 60% file và chỉ giữ statement trong cùng (không bôi cả hàm).
- CPG-GNN bắt cặp dòng trên vector node GAT.
- MOSS tô đúng vùng khớp mà server Stanford trả về trên trang match (cặp không được gắn cờ hiện trắng trơn, không có màu nào).
- Vàng lẻ là token mô hình dựa vào để cho điểm nhưng không ghép được cặp, không màu là phần còn lại.

6.1 Cặp clone khó: P1_combo (bộ test custom_suite)

Cùng bài tổng 1..n, nhưng bản B viết lại toàn bộ: `for` thành `while`, `cin/cout` thành `scanf/printf`, đổi tên biến, đảo điều kiện phủ định, thêm biến thừa. Cả 5 phương pháp kết luận đúng: M1 1,000, M2 1,000, M3 0,812, M4 0,915, M5 1,000, đều trên ngưỡng nên là clone. MOSS server mù với biến thể này (0,000) — đúng hạn chế đã biết ở mục 6 của bản trước: server chỉ trả top match nên cặp Type-3/4 viết lại kỹ không được gắn cờ.

CodeBERT — 1,000 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B (p1_v_combo)

```
#include <cstdio>
int main() {
int N;
scanf("%d", &N);
int acc = 0;
int step = 1;
int unused = 999;
(void)unused;
int k = 1;
while (!(k > N)) {
acc = acc + k;
k = k + step;
}
printf("%d\n", acc);
return 0;
}
```

GraphCodeBERT — 1,000 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B (p1_v_combo)

```
#include <cstdio>
int main() {
int N;
scanf("%d", &N);
int acc = 0;
int step = 1;
int unused = 999;
(void)unused;
int k = 1;
while (!(k > N)) {
acc = acc + k;
k = k + step;
}
printf("%d\n", acc);
return 0;
}
```

ASTNN — 0,812 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B (p1_v_combo)

```
#include <cstdio>
int main() {
int N;
scanf("%d", &N);
int acc = 0;
int step = 1;
int unused = 999;
(void)unused;
int k = 1;
while (!(k > N)) {
acc = acc + k;
k = k + step;
}
printf("%d\n", acc);
return 0;
}
```

CPG-GNN — 0,915 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
  int n, sum = 0;
  cin >> n;
  for (int i = 1; i <= n; i++) {
  sum += i;
}
cout << sum << endl;
  return 0;
}
```

File B (p1_v_combo)

```
#include <cstdio>
int main() {
  int N;
  scanf("%d", &N);
int acc = 0;
  int step = 1;
int unused = 999;
(void)unused;
int k = 1;
  while (!(k > N)) {
  acc = acc + k;
  k = k + step;
}
printf("%d\n", acc);
  return 0;
}
```

Hybrid — 1,000 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std ;
int main () {
int n, sum = 0 ;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B (p1_v_combo)

```
#include <cstdio>
int main () {
int N;
scanf ("%d", &N);
int acc = 0 ;
int step = 1;
int unused = 999 ;
( void)unused ;
int k = 1;
while (!(k > N)) {
acc = acc + k ;
k = k + step ;
}
printf ("%d\n", acc);
return 0;
}
```

MOSS (server Stanford) — 0,000 (không clone)

File A

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B

```
#include <cstdio>
int main() {
int N;
scanf("%d", &N);
int acc = 0;
int step = 1;
int unused = 999;
(void)unused;
int k = 1;
while (!(k > N)) {
acc = acc + k;
k = k + step;
}
printf("%d\n", acc);
return 0;
}
```

6.2 Cặp không clone khó: P3_nonclone (bộ test custom_suite)

Cùng skeleton đọc mảng bằng `for` và in một số, nhưng một bên tìm `max`, bên kia tính tổng. M1 0,000, M2 0,110 và M5 0,005 dưới ngưỡng nên kết luận đúng là không clone; M3 0,852 và M4 0,973 trên ngưỡng nên kết luận sai. MOSS server không gắn cờ (0,000) nên đúng ở cặp này. Cặp này cho thấy bẫy skeleton giống hệt: mô hình cấu trúc (ASTNN, CPG-GNN) dễ bị đánh lừa hơn mô hình transformer.

CodeBERT — 0,000 (không clone)

File A (p3_base_max)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
int mx = INT_MIN, x;
for (int i = 0; i < n; i++) {
cin >> x;
if (x > mx) mx = x;
}
cout << mx << endl;
return 0;
}
```

File B (p3_n_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long total = 0, x;
for (int i = 0; i < n; i++) {
cin >> x;
total += x;
}
cout << total << endl;
return 0;
}
```

GraphCodeBERT — 0,110 (không clone)

File A (p3_base_max)

```
#include <bits/stdc++.h>
using namespace std ;
int main () {
int n;
cin >> n;
int mx = INT_MIN, x;
for (int i = 0; i < n; i++) {
cin >> x;
if (x > mx) mx = x;
}
cout << mx << endl ;
return 0;
}
```

File B (p3_n_sum)

```
#include <bits/stdc++.h>
using namespace std ;
int main () {
int n;
cin >> n;
long long total = 0, x;
for ( int i = 0; i < n; i++) {
cin >> x;
total += x;
}
cout << total << endl ;
return 0;
}
```

ASTNN — 0,852 (clone)

File A (p3_base_max)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
int mx = INT_MIN, x;
for (int i = 0 ; i < n; i++) {
 cin >> x;
 if (x > mx) mx = x;
 }
cout << mx << endl;
return 0;
}
```

File B (p3_n_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long total = 0 , x;
for (int i = 0; i < n; i++) {
 cin >> x;
 total += x;
 }
cout << total << endl;
return 0;
}
```

CPG-GNN — 0,973 (clone)

File A (p3_base_max)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 int n;
 cin >> n;
 int mx = INT_MIN, x;
 for (int i = 0; i < n; i++) {
 cin >> x;
if (x > mx) mx = x;
}
 cout << mx << endl;
 return 0;
}
```

File B (p3_n_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 int n;
 cin >> n;
 long long total = 0, x;
 for (int i = 0; i < n; i++) {
 cin >> x;
total += x;
}
 cout << total << endl;
 return 0;
}
```

Hybrid — 0,005 (không clone)

File A (p3_base_max)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
int mx = INT_MIN, x;
for (int i = 0; i < n; i++) {
cin >> x;
if (x > mx) mx = x;
}
cout << mx << endl;
return 0;
}
```

File B (p3_n_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long total = 0, x;
for (int i = 0; i < n; i++) {
cin >> x;
total += x;
}
cout << total << endl;
return 0;
}
```

MOSS (server Stanford) — 0,000 (không clone)

File A

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
int mx = INT_MIN, x;
for (int i = 0; i < n; i++) {
cin >> x;
if (x > mx) mx = x;
}
cout << mx << endl;
return 0;
}
```

File B

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long total = 0, x;
for (int i = 0; i < n; i++) {
cin >> x;
total += x;
}
cout << total << endl;
return 0;
}
```

6.3 Cặp bẫy skeleton: P1_nonclone_fact (cả 5 model đều sai)

Cùng skeleton vòng `for` 1..n và in một số, nhưng một bên tính tổng, bên kia tính giai thừa (tích). Cả 5 mô hình neural đều kết luận sai là clone: M1 1,000, M2 1,000, M3 0,960, M4 0,971, M5 1,000, đều trên ngưỡng. MOSS server cho 0,000 nên đúng ở cặp này, nhưng là đúng do mù (nó cũng cho 0,000 với cặp clone khó ở mục 6.1) chứ không phải do hiểu code. Cặp `p4_n_square` (số nguyên tố so với số chính phương, cùng khung `for` trả lời YES/NO) cũng khiến cả 5 model sai tương tự nên không vẽ lại hình. Ngược lại, chiều sai không clone (GT là clone mà cả 5 model đều bảo không clone) không xuất hiện trong bộ test: cả 16 cặp clone, kể cả 4 cặp combo khó nhất, đều được 5 model bắt đúng.

CodeBERT — 1,000 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B (p1_n_fact)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long prod = 1;
for (int i = 1; i <= n; i++) {
prod *= i;
}
cout << prod << endl;
return 0;
}
```

GraphCodeBERT — 1,000 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std ;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1 ; i <= n ; i ++) {
sum += i;
}
cout << sum << endl ;
return 0;
}
```

File B (p1_n_fact)

```
#include <bits/stdc++.h>
using namespace std ;
int main() {
int n ;
cin >> n;
long long prod = 1;
for (int i = 1 ; i <= n ; i ++) {
prod *= i;
}
cout << prod << endl;
return 0;
}
```

ASTNN — 0,960 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0 ;
cin >> n;
for ( int i = 1; i <= n; i++) {
  sum += i;
  }
cout << sum << endl;
return 0;
}
```

File B (p1_n_fact)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long prod = 1 ;
for ( int i = 1; i <= n; i++) {
  prod *= i;
  }
cout << prod << endl;
return 0;
}
```

CPG-GNN — 0,971 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
  int n, sum = 0;
  cin >> n;
  for (int i = 1; i <= n; i++) {
  sum += i;
}
  cout << sum << endl;
  return 0;
}
```

File B (p1_n_fact)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
  int n;
  cin >> n;
  long long prod = 1;
  for (int i = 1; i <= n; i++) {
  prod *= i;
}
  cout << prod << endl;
  return 0;
}
```

Hybrid — 1,000 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B (p1_n_fact)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long prod = 1;
for (int i = 1; i <= n; i++) {
prod *= i;
}
cout << prod << endl;
return 0;
}
```

MOSS (server Stanford) — 0,000 (không clone)

File A

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long prod = 1;
for (int i = 1; i <= n; i++) {
prod *= i;
}
cout << prod << endl;
return 0;
}
```

7 Hạn chế

- Hybrid đỉnh ngay epoch đầu rồi overfit, CPG-GNN thì val F1 còn tăng ở epoch cuối.
- M1/M2 bản 256 cắt đầu vào 256 token (78,6% train bị cắt), CPG-GNN chặn 1500 node, hàm dài bị mất thông tin. Bản 512 đã giảm tỉ lệ cắt xuống 25,5% và F1 tăng +0,031/+0,048 (mục 3b), nhưng 1/4 code dài nhất vẫn bị cắt.

8 Kế hoạch tiếp theo

- Hybrid V2 (giảm LR) không tốt hơn V1 nên giữ V1; tiếp theo thử freeze encoder, chỉ train head rồi mới unfreeze.
- Đầu vào 512 token đã thử xong cho M1/M2 (cải thiện rõ); tiếp theo nới giới hạn node của CPG-GNN và thử Hybrid trên nền encoder 512.